```python
"""
Project 2: Data Classification Using AI
Goal: Build a basic classification model using a small dataset.

Dataset: Iris flower dataset (built into scikit-learn) - 150 samples,
3 classes (Setosa, Versicolor, Virginica), 4 numeric features.
Algorithm: Decision Tree Classifier (simple, interpretable).
"""

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matr
import pandas as pd


def load_and_understand_data():
    """Load the dataset and print basic info to understand it."""
    iris = load_iris()
    df = pd.DataFrame(iris.data, columns=iris.feature_names)
    df["target"] = iris.target
    df["species"] = df["target"].apply(lambda i: iris.target_names[i])

    print("=" * 50)
    print(" STEP 1: Load and Understand the Dataset")
    print("=" * 50)
    print(f"Shape: {df.shape[0]} rows, {df.shape[1]} columns")
    print("\nFirst 5 rows:")
    print(df.head())
    print("\nClass distribution:")
    print(df["species"].value_counts())
    print("\nBasic statistics:")
    print(df.describe())

    return iris, df


def split_data(iris):
    """Split data into training and testing sets."""
    print("\n" + "=" * 50)
    print(" STEP 2: Split Data into Training and Testing Sets")
    print("=" * 50)

    X = iris.data
```

```
    y = iris.target

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y
    )

    print(f"Training samples: {X_train.shape[0]}")
    print(f"Testing samples: {X_test.shape[0]}")

    return X_train, X_test, y_train, y_test


def train_and_classify(X_train, X_test, y_train, y_test, target_names):
    """Apply a simple classification algorithm (Decision Tree)."""
    print("\n" + "=" * 50)
    print(" STEP 3: Apply a Simple Classification Algorithm")
    print("=" * 50)

    model = DecisionTreeClassifier(random_state=42)
    model.fit(X_train, y_train)

    predictions = model.predict(X_test)

    accuracy = accuracy_score(y_test, predictions)
    print(f"\nModel: Decision Tree Classifier")
    print(f"Accuracy on test set: {accuracy:.2%}")

    print("\nClassification Report:")
    print(classification_report(y_test, predictions, target_names=target_names))

    print("Confusion Matrix:")
    print(confusion_matrix(y_test, predictions))

    return model


def predict_new_sample(model, target_names):
    """Demonstrate predicting on a brand-new, unseen sample."""
    print("\n" + "=" * 50)
    print(" BONUS: Predict a New Sample")
    print("=" * 50)

    # Example measurements: sepal length, sepal width, petal length, petal width
    new_sample = [[5.1, 3.5, 1.4, 0.2]]
    prediction = model.predict(new_sample)
    species = target_names[prediction[0]]
```

```
    print(f"New sample measurements: {new_sample[0]}")
    print(f"Predicted species: {species}")


def main():
    iris, df = load_and_understand_data()
    X_train, X_test, y_train, y_test = split_data(iris)
    model = train_and_classify(X_train, X_test, y_train, y_test, iris.target_name
    predict_new_sample(model, iris.target_names)


if __name__ == "__main__":
    main()
```